Práctica 4. Estructuras STL

Sesiones de prácticas: 2

***Importante**: esta práctica y las siguientes deben ser realizadas por los estudiantes que convalidaron las tres primeras prácticas.*

Objetivo

Aprender a utilizar estructuras de datos de la Standard Template Library (STL).

Descripción de la EEDD

En esta práctica reemplazamos las EEDD implementadas en sesiones anteriores por contenedores de STL.

Descripción de la práctica

En esta nueva práctica vamos a sustituir las estructuras de datos iniciales por las de STL. Por tanto, usaremos *vector* para las *Especie*, *list* para los *Parque* y *map* para los *Arbol*. Además, se añaden dos nuevas relaciones:

- Una relación de asociación de *Arbol* a *Parque Arbol::isInto*; esta nueva conexión se realiza al mismo tiempo que se enlaza el *Parque* con el *Arbol.*
- Una nueva relación de asociación entre *ControlParques* y *Arbol*, *ControlParques::treeName*; esto permitirá buscar eficientemente los árboles dado el nombre común de la especie a la que pertenecen y con ello, localizar los parques en los que se encuentran. Esta relación se define mediante un *multimap* donde la clave es el nombre común de la especie.

El resto de la funcionalidad es similar a la práctica anterior, aunque considerando los siguientes cambios:

- Se añaden estas funciones a *ControlParques*:
  - *listarParquesArbol()*: devuelve un conjunto de parques que tienen al menos un ejemplar del árbol dado por parámetro mediante su nombre común.

    - *buscarArbolNcomun():* localiza en tiempo óptimo todos los ejemplares de árboles con ese nombre común exacto, p.e. “Pino laricio”.
    - *buscarArbolNcomunComienzo():* localiza en tiempo óptimo todos los ejemplares de árboles dando una palabra del comienzo, p.e. si damos “Pino” también obtenemos *Pino laricio* y *Pino resinero.* Para obtener este resultado usaríamos “lower_bound”.
    - *contarArbolesBarrio()*: devuelve el número de árboles que tiene un barrio de Madrid dado su código, *num_barrio*.
- En la clase *Arbol* se añaden las siguientes funciones debido a la nueva conexión:
    - *setParque()*: enlaza un árbol con un parque a través de *Arbol::isInto,*
    - *getParque()*: devuelve el parque en el que se encuentra.
- La clase *Parque* añade:
    - *especieMasComun ():* devuelve la especie que más ejemplares tiene en ese parque. Si hay empate, devolver la primera que alcanza la máxima frecuencia.

El UML de la nueva funcionalidad se refleja a continuación:



Resumiendo, los contenedores a usar para las cinco relaciones “uno a muchos” son:

- ControlParques::parks → std::list<Parque>
- ControlParques::trees → std::map<string, Arbol> con clave assetnum
- ControlParques::varieties → std::vector<Especie>
- ControlParques::treeName → std::multimap<string, Arbol*> con clave nbre_comun
- Parque::contains → std::vector<Arbol*>

A continuación se describen los pasos que debe seguir el **constructor de la clase ControlParques**, que tomará los nombres de los 3 ficheros como parámetros:

1. Leer el fichero de especies y almacenar los objetos *Especie* en un vector.
2. Leer el fichero de parques y almacenar los objetos *Parque* en una lista.
3. Leer el fichero «arboladoMadrid.csv» e insertar los árboles en un *map*, donde la clave es *assetnum*. Asociar cada árbol con su especie mediante CODIGO_ESPECIE y establecer la asociación entre el parque correspondiente, identificado mediante NUM_PARQUE, y el árbol almacenado en el *map*. A su vez, establecer el parque al que pertenece cada árbol mediante *Arbol::isInto* y el *multimap* de árboles con clave el nombre común de la especie.

Programa de prueba:

1. Buscar en el mapa los árboles cuyos identificadores son 6951831, 10152826, 8887777 y 6906686. Mostrar su identificador, código de parque, nombre común, nombre científico y tipo de planta, si existe.
2. Mostrar cuál es la especie más común en el parque “PARQUE PRINCESA LEONOR”.
3. Busca todos aquellos parques que tienen al menos un árbol cuyo nombre común es “abeto”. Muestra toda la información de los parques.
4. Busca los ejemplares de árbol cuyo nombre común sea “Pino resinero” y todos los ejemplares que empiecen por “Pino” .
5. ¿Cuántos árboles tiene el barrio con número 43? ¿Y el barrio con número 91?

Para los que trabajan en parejas:

- Añadid a la clase ControlParques un método que devuelva un listado de los árboles de una especie dada por su nombre común y que pertenecen a un mismo distrito:

  buscarArbolesPorEspecieYDistrito(nbreComun, numDistrito);
- Llamad al método para buscar todos los árboles con nombre común "Pino" ubicados en el distrito 9.
- Mostrar el identificador del árbol (assetnum), el nombre del barrio (nbre_barrio) y el nombre del parque en el que se encuentra (accediendo a través del enlace Arbol::isInto).

IMPORTANTE:

- **Cuidado con realizar copias de objetos**
- **Añadid *getters* y *setters* cuando se necesiten**